

S3. 스켈레톤 코드 구조 + 의존 방향

모노레포 · 패키지 지도 · Dependency Rule · Strategy Pattern

2026.05 | M1 S3

멀티레포 vs 모노레포 — 두 가지 전략의 특징

멀티레포 (Multi-repo)

특징

- 서비스별로 레포지토리를 분리
- 팀·배포 단위가 명확히 구분
- 레포마다 독립적인 CI/CD·권한 설정 가능

잘 맞는 상황

- 팀이 크고 서비스가 독립적으로 운영
- 서비스 간 배포 주기·기술 스택이 다름
- 레포별 접근 권한을 엄격하게 분리해야 함
- ex) Netflix, Amazon — 수백 개 독립 서비스

모노레포 (Mono-repo)

특징

- 여러 패키지를 하나의 레포에서 관리
- 패키지 간 코드·타입을 직접 공유
- 전체 코드베이스를 한 번에 빌드·테스트

잘 맞는 상황

- 패키지 간 타입·인터페이스를 자주 공유
- 작은 팀이 여러 패키지를 함께 관리
- 인터페이스 변경이 전체에 즉시 반영되어야 함
- ex) Google, Meta 내부 도구, TypeScript 모노레포

어느 쪽이 '더 좋다'는 없다. 팀 규모·서비스 결합도·배포 독립성에 따라 선택.

이 프로젝트에서 모노레포를 선택한 이유

① 패키지 간 결합도가 높다

6개 패키지가 동일한 인터페이스를 공유하며 하나의 서비스를 구성.

인터페이스 변경이 전체에 동시 반영되어야 하므로 원자적 커밋이 필요하다.

→ 모노레포의 단일 커밋이 자연스럽게 맞아떨어짐

② 타입 안전성이 즉시 필요

TypeScript는 컴파일 타임에 전체 패키지를 한 번에 체크.

npm run typecheck
→ IBlockchainAdapter 변경 시
EVMAdapter,
VaspRecoveryService
컴파일 에러 즉시 발생

→ 멀티레포였다면 배포 후 발견

③ 소수 팀이 전체 코드를 관리

레포 간 접근 권한·CI 설정을 분리해야 할 이유가 없다.

패키지별 배포 주기도 동일.
(issuer-service 하나로 묶여서 배포)

→ 멀티레포의 격리 비용 대비
얻는 이점이 없음

결론: 이 프로젝트에선 '결합도 높은 패키지들 + 소수 팀 + 단일 배포 단위' → 모노레포가 적합.

npm Workspaces — 모노레포가 해결하는 것

npm Workspaces 모노레포의 3가지 해결

- ① 최상위에서 npm install 하면 모든 패키지 한 번에 설치
→ 패키지 간 이동 없이 단일 작업 흐름
- ② 패키지 간 import가 패키지 이름으로 가능 (@kyobo/vasp)
→ npm link 불필요. IDE 자동완성·타입 즉시 동작
- ③ 전체 타입체크를 한 번에 실행 (npm run typecheck)
→ 인터페이스가 바뀌면 전 패키지 컴파일 에러 즉시 발견

타입 안전성 = 모노레포의 핵심 가치. 인터페이스 변경이 전체에 즉시 전파.

npm Workspaces — 모노레포 구현 방식

루트 package.json 설정

```
{
  "name": "kyobo-digital-asset-platform",
  "workspaces": [
    "dmz/apps/*",
    "dmz/packages/*",
    "blockchain"
  ]
}
```

→ npm install 한 번으로 전체 설치
→ node_modules/@kyobo/* 자동 심링크

실제로 얻는 것

- import { IVASPAAdapter } from '@kyobo/vasp'
→ npm link 없이 바로 동작
- npm run typecheck (루트)
→ 전 패키지 컴파일 에러 즉시 발견
- 단일 package-lock.json
→ 버전 일관성 자동 보장
- 인터페이스 변경 → 단일 커밋으로 처리

도구가 전략을 강제하지 않는다. 선택한 전략(모노레포)에 맞는 도구(npm workspaces)를 골랐을 뿐.

@kyobo/* 네임스페이스 — 어떻게 동작하나

각 패키지 package.json — name 필드

```
dmz/packages/vasp
→ "name": "@kyobo/vasp"
dmz/packages/core-banking
→ "name": "@kyobo/core-banking"
dmz/packages/chain-adapters
→ "name": "@kyobo/chain-adapters"
```

실제 import 예시

```
// issuer-service에서
import { IVASPAdapter }
  from '@kyobo/vasp'
import { LedgerService }
  from '@kyobo/core-banking'
```

node_modules/@kyobo/vasp → dmz/packages/vasp/src 심링크. 경로 외울 필요 없음.

통합 typecheck — 전체 패키지를 한 번에

실행 방법

```
# 루트에서  
npm run typecheck
```

```
# 내부적으로  
tsc --noEmit (각 패키지 순서대로)
```

왜 중요한가

- IBlockchainAdapter.mintNFT 시그니처 변경
 - EVMAdapter 컴파일 에러 즉시 발생
 - VaspRecoveryService 컴파일 에러 즉시
 - 빌드 전에 전파 범위 파악 가능

멀티레포였다면? 배포 후 런타임에야 발견. 모노레포는 빌드 단계에서 차단.

레포 3영역 — 언어 · 환경 · 신뢰 수준으로 분리

blockchain/
Solidity / EVM
Hardhat 환경
→ M6 컨트랙트

TypeScript import 

dmz/
TypeScript / Node.js
ISMS-P DMZ
→ M2~M5 핵심

TypeScript import 

internal/
Java / Spring Boot
교보 내부망
→ M4 원장·감사

HTTP 전용 — import 

dmz/ — TypeScript / Node.js 서비스 영역

위치와 역할

- ISMS-P DMZ 구간 배포
- TypeScript / Node.js 서비스
- 외부(VASP) ↔ 내부망 사이 중간 레이어
- M2~M5 실습의 핵심 영역

구성

dmz/packages/ → 6개 공유 패키지
dmz/apps/ → 1개 서비스 앱

packages = 재사용 가능 라이브러리
apps = 실제 실행 서버

dmz/ 전체가 npm workspace 하나. @kyobo/* import로 자유롭게 연결.

dmz/packages/ — 6개 패키지 역할 한눈에

패키지	역할	배우는 곳
chain-adapters	IBlockchainAdapter — EVM/Circle/XRPL 체인 추상화	M3
vasp	IVASPAdapter — 월렛원 등 VASP 추상화	M3
core-banking	ICoreBankingAdapter — 내부망 RPC + Ledger	M4
event-engine	Redis Streams 이벤트 파이프라인 + Webhook	M2
compliance	IKYCProvider — KYC·AML·ISMS-P 컴플라이언스	M7
shared	공통 타입·에러·설정 (의존 방향 최하단)	전 모듈

issuer-service(app)가 이 6개 패키지를 모두 import해서 사용.

blockchain/ — Solidity 컨트랙트 영역

역할

- Solidity 스마트 컨트랙트 소스
- Hardhat 개발 환경 (컴파일·배포·테스트)
- EVM 기반 (Sepolia 테스트넷 → Mainnet)

dmz와의 연결

- typechain이 ABI에서 TS 타입 자동 생성
- chain-adapters에서 typechain 타입 import
- 런타임 연결: ethers.js → RPC URL

blockchain/은 M6에서 집중적으로 배움. M1~M5는 스켈레톤 수준으로만 이해.

blockchain/ — 파일 구조 상세

src/ — 컨트랙트

src/reward/KyoboNFT.sol ← M6 핵심
 src/reward/NFTIssuer.sol
 src/reward/ActivityOracle.sol
 src/stablecoin/KRWStablecoin.sol (placeholder)
 src/securities/SecurityToken.sol (placeholder)
 src/base/BaseToken.sol
 src/compliance/InvestorCompliance.sol
 src/compliance/PermissiveCompliance.sol
 src/interfaces/ — IToken, ICompliance
 ISecurityToken, IDividendDistributor
 IInvestorRegistry, IOracle
 src/mocks/MockERC1155.sol ← Sepolia
 테스트용

빌드·배포·테스트

scripts/deploy/deploy-phase1.ts ← M6 배포
 scripts/deploy/deploy-phase3.ts (placeholder)
 scripts/deploy/deploy-mock-sepolia.ts
 test/ ← M6/M7
 hardhat.config.ts

 npx hardhat compile → artifacts/
 npx hardhat test → HH Network 실행
 npx hardhat node → localhost:8545

phase2/3, 새 인터페이스들 = placeholder/stub. M1에서는 phase1 파일만 집중.

internal/ — Java Spring Boot 내부망 서비스

controller/ · service/

BlockchainGatewayController.java
GET /api/internal/users/{userId}
POST /api/internal/users/{userId}/nft-holdings
POST /api/internal/audit-log
POST /api/internal/rewards/notify
AuditLogService.java ← SHA-256 감사 체인
InternalLedgerService.java ← NFT 보유 원장

entity/ · repository/ · dto/

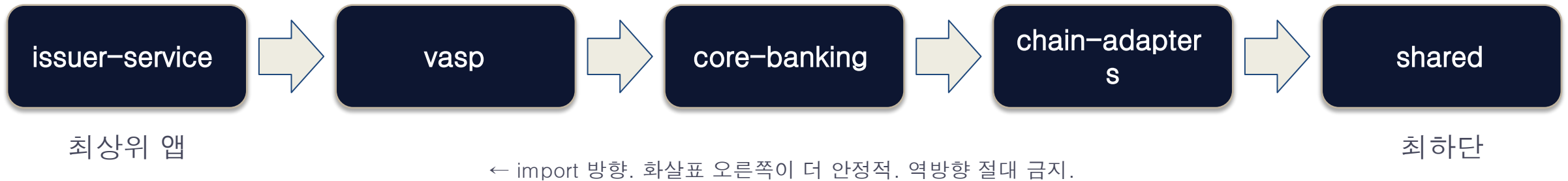
AuditLogEntry.java (append-only + RLS)
NftHolding.java (사용자별 NFT 보유)
AuditLogRepository ← INSERT only
NftHoldingRepository
NftHoldingRequest.java ← Java 17 record
UserAccountResponse.java

TS에서 import 불가 — HTTP JSON DTO만. Java 17 / Spring Boot 3.2.4.

레이어 → 패키지 매핑 — 어디서 무엇을 배우나

S2 레이어	패키지 / 앱	모듈
레이어 1 — 내부망 원장	internal/blockchain-gateway	M4
레이어 1 — DMZ 비즈니스	dmz/apps/issuer-service	M5
레이어 2 — 이벤트 파이프	dmz/packages/event-engine	M2
레이어 2 — VASP 추상화	dmz/packages/vasp	M3
레이어 2 — 체인 추상화	dmz/packages/chain-adapters	M3
레이어 2 — 내부망 RPC	dmz/packages/core-banking	M4
레이어 4 — 스마트 컨트랙트	blockchain/	M6

패키지 간 import 관계



blockchain/ import

chain-adapters → blockchain/ typechain 타입 import 가능 (ABI 타입)
→ 컨트랙트 ABI가 바뀌면 chain-adapters 컴파일 에러 즉시 발생

shared에서 오른쪽으로 의존하는 것 = 금지.

chain-adapters — 역할과 위치

역할

- IBlockchainAdapter 인터페이스 정의
- EVMAdapter (M3 핵심 구현체)
- 체인 교체 포인트 — 이 패키지만 수정

Phase 1 실제 사용

- getBalance, queryEvents 주 용도
- mintNFT는 구현됨 (월렛원이 실제 실행)
- EVMAdapter = ethers.js v6 기반
- Sepolia 테스트넷 연결

체인을 XRPL로 바꾸고 싶으면? chain-adapters만 수정. 상위 레이어 무변경.

chain-adapters — 파일 구조

interfaces/

IBlockchainAdapter.ts ← M3 핵심 계약
ChainEvent.ts ← 이벤트 타입
MintParams.ts ← 발행 파라미터
TransactionReceipt.ts ← 영수증 타입

evm/ · circle/ · xrpl/ · utxo/

evm/EVMAdapter.ts ← M3 EVM 구현체
(ethers.js v6 기반)

circle/CircleAdapter.ts ← Circle CCTP 연동
xrpl/XRPLAdapter.ts ← Phase 2 stub
utxo/UTXOAdapter.ts ← stub

M3 실습은 EVMAdapter.ts와 IBlockchainAdapter.ts가 핵심. Circle/XRPL/UTXO는 stub.

chain-adapters — IBlockchainAdapter 주요 메서드

메서드	역할
mintNFT(p: MintParams)	NFT 1개 발행 → TransactionReceipt
mintNFTBatch(p: MintBatchParams)	NFT 다건 발행 → TransactionReceipt
burnNFT(p: BurnParams)	NFT 소각
getBalance(addr, op, tokenId)	ERC-1155 잔액 조회 → bigint
queryEvents(addr, topics, ...)	과거 이벤트 조회 → ChainEvent[]
subscribeEvents(..., handler)	실시간 이벤트 구독 → unsubscribe 함수

S14 실습: 이 메서드들을 EVM/XRPL/Circle/UTXO 4개 어댑터로 구동.

vasp — VASP 추상화 레이어

역할

- IVASPAdapter 인터페이스 정의
- ExternalVASPAdapter (월렛원 연동)
- VASP 교체 포인트 — 이 패키지만 수정
- Idempotency-Key, requestId 관리

Phase 1 실제 사용

- ExternalVASPAdapter = 월렛원 API 호출
- TxStateMachineService = TX 상태 추적
- KyoboVASPAdapter = Phase 3 내재화 stub

VASP를 Fireblocks로 바꾸고 싶으면? vasp만 수정. issuer-service 무변경.

vasp — 파일 구조

interfaces/ · external/

interfaces/IVASPAdapter.ts ← M3 핵심
external/ExternalVASPAdapter.ts ← 월렛원 연동
internal/KyoboVASPAdapter.ts ← Phase 3 stub

tx/ · recovery/ · governance/

tx/TxStateMachineService.ts ← M3 상태머신
recovery/VaspRecoveryService.ts ← M3 복구
governance/KeyGovernanceService.ts ← M8 키
거버넌스

governance/ = M8. M3에서는 인터페이스와 ExternalVASPAdapter만 집중.

TxStateMachineService — TX 상태 추적

TX 상태 흐름

PENDING → SUBMITTED → CONFIRMED
↓ (실패 시)
FAILED → RECOVERY 시도

왜 필요한가

- 블록체인 TX는 결과가 비동기
- 네트워크 장애 시 TX 유실 가능
- 상태머신이 재시도·복구 보장
- Idempotency-Key로 중복 방지

M3 실습: TX 상태가 CONFIRMED될 때까지 폴링 또는 이벤트 구독.

core-banking — 내부망 RPC + Ledger

역할

- ICoreBankingAdapter 인터페이스 정의
- InternalGatewayClient (HTTP 클라이언트)
- LedgerService (mint_requests 상태머신)
- AuditLogService (TX 이벤트 로그)

두 구현체

KyoboCoreBankingAdapter → 운영 (Java HTTP)
StubCoreBankingAdapter → 테스트 (인메모리)

→ IssuerService는 어느 쪽인지 모름

DMZ(TS) ↔ 내부망(Java) 계약. 언어가 달라도 인터페이스로 타입 안전성 유지.

core-banking — 파일 구조

interfaces/ · adapters/

interfaces/ICoreBankingAdapter.ts ← 계약
adapters/InternalGatewayClient.ts ← HTTP
클라이언트
adapters/KyoboCoreBankingAdapter.ts ← 운영용
adapters/StubCoreBankingAdapter.ts ←
테스트·로컬
adapters/KDEPAdapter.ts ← Phase 3 stub

ledger/ · audit/ · reconcile/

ledger/LedgerService.ts ← M4 상태머신
audit/AuditLogService.ts ← M4 감사 로그
reconcile/ReconcileService.ts ← M4 대사

ledger, audit, reconcile = M4의 핵심. M1에서는 구조 파악만.

LedgerService · AuditLogService — M4 핵심 서비스

LedgerService

- mint_requests 테이블 상태머신 관리
- 상태: PENDING → PROCESSING → DONE
- idempotency_key로 중복 요청 차단
- M4 실습에서 직접 구현

AuditLogService (DMZ 범위)

- TX 이벤트 로그 기록
- SHA-256 체인 해시 (변조 방지)
- 내부망 AuditLogService와 분리
- DMZ 범위 감사 추적

내부망(Java) AuditLogService와 DMZ(TS) AuditLogService — 이름 같지만 별개.

StubCoreBankingAdapter — 테스트 전용 검증 메서드

인메모리 저장소

```
private holdings: unknown[] = [];  
private auditLog: unknown[] = [];  
  
async recordNftHolding(p) {  
  this.holdings.push(p);  
}  
async recordAuditLog(e) {  
  this.auditLog.push(e);  
}
```

검증 메서드 (ICoreBankingAdapter 외부)

```
// 테스트에서만 호출  
stub.getHoldings()  
→ recordNftHolding() 호출 이력  
  
stub.getAuditLog()  
→ recordAuditLog() 호출 이력  
  
assert(stub.getAuditLog().length === 1)
```

Stub 검증 메서드는 ICoreBankingAdapter에 없음. Stub 전용 API.

event-engine — DMZ 이벤트 파이프라인

역할

- Redis Streams 기반 이벤트 처리
- Webhook 202 즉시 반환 패턴
- 멍등성 보장 (IdempotencyGuard)
- DLQ (Dead Letter Queue) 처리

위치

- dmz/packages/event-engine
- M2 실습이 이 패키지 안에서 전부
- ConsumerGroupWorker → XREADGROUP
- ChainEventListener → 체인 이벤트 구독

202 패턴 → Webhook이 즉시 응답. 실제 처리는 Consumer가 비동기로.

event-engine — 파일 구조

dmz/ · webhook/

dmz/ConsumerGroupWorker.ts ← XREADGROUP
루프

dmz/DLQHandler.ts ← DLQ 처리

dmz/RedisStreamPublisher.ts ← XADD 발행

webhook/WebhookServer.ts ← 202 즉시 반환

webhook/IdempotencyGuard.ts ← 중복 차단

webhook/RetryHandler.ts ← 재시도

listener/ · handlers/

listener/ChainEventListener.ts ← 이벤트 구독

handlers/NFTIssuedHandler.ts ← 이벤트 처리

M2 실습: 이 파일들을 순서대로 완성하면 전체 파이프라인 동작.

S2 개념 → event-engine 구현체 매핑

S2 개념	event-engine 구현체
202 즉시 반환 패턴	webhook/WebhookServer.ts
Redis Streams XADD	dmz/RedisStreamPublisher.ts
XREADGROUP Consumer	dmz/ConsumerGroupWorker.ts
PEL / XACK 처리	dmz/ConsumerGroupWorker.ts
DLQ 이관	dmz/DLQHandler.ts
멥등성 보장	webhook/IdempotencyGuard.ts

S2에서 배운 모든 패턴이 이 패키지 안에서 구현된다.

issuer-service — 모든 패키지를 조합하는 최상위 앱

issuer-service의 위치

- dmz/apps/issuer-service
- 모든 패키지를 import해 조합
- 비즈니스 로직의 집결점
- M5까지 배우면 이 앱이 완성됨

import 관계

```
import from '@kyobo/vasp'  
import from '@kyobo/core-banking'  
import from '@kyobo/event-engine'  
import from '@kyobo/chain-adapters'  
import from '@kyobo/shared'
```

M1~M4 = 부품 제작. M5 = 조립. issuer-service가 최종 결과물.

issuer-service — 파일 구조

services/

IssuerService.ts ← M5 단건 발행
BulkIssueService.ts ← M5 벌크 발행
EventConditionService.ts ← M5 조건 판단
WalletMappingService.ts ← M5 지갑-사용자 매핑
WalletProvisioningService.ts ← M5 VASP별 지갑
프로비저닝

api/ · factory/

api/ActivityRouter.ts ← M5 REST API
factory/TokenIssuerFactory.ts ← M5 팩토리

팩토리: 환경에 따라 Stub vs 실제 구현체 선택

TokenIssuerFactory가 어떤 어댑터를 주입할지 결정. 나머지는 인터페이스만 봄.

shared — 공통 기반 · 의존 방향 최하단

shared에 있는 것

src/types/ ← 공통 DTO · 인터페이스
src/errors/ ← 공통 에러 클래스
src/config/ ← 공통 설정

모든 패키지가 shared를 import 가능

shared의 규칙

- shared는 아무것도 import 안 함
- 의존 방향 최하단 → 순환 참조 불가
- 공통 타입 → 반드시 여기에 추가
- 'vasp에서 조금만...' → 안 됨

shared에서 다른 패키지를 import하면 → Dependency Rule 위반 → 순환 참조 시작.

Dependency Rule

의존성은 항상 안쪽(더 안정적인 레이어)으로만

바깥 → 안쪽 허용

안정적인 레이어 (안쪽)

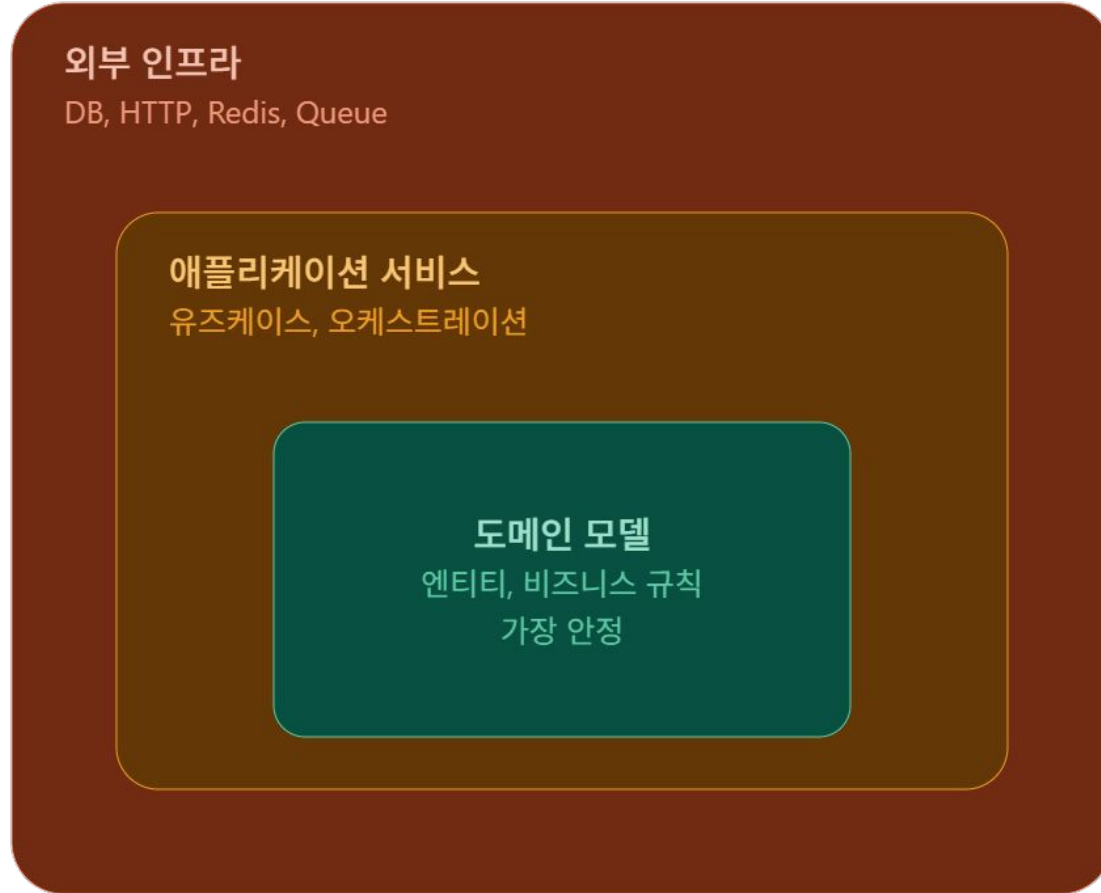
- 인터페이스 (거의 안 변함)
- shared 타입 (매우 안정)
- chain-adapters 인터페이스

안쪽 → 바깥 금지

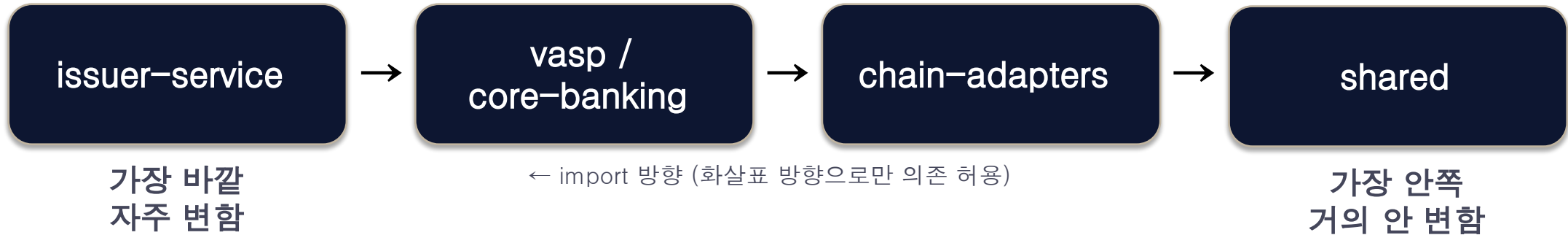
자주 변하는 레이어 (바깥)

- issuer-service (요구사항 변화)
- EventConditionService
- 비즈니스 로직 전반

Dependency Rule



TypeScript import 의존 방향 — 컴파일 타임

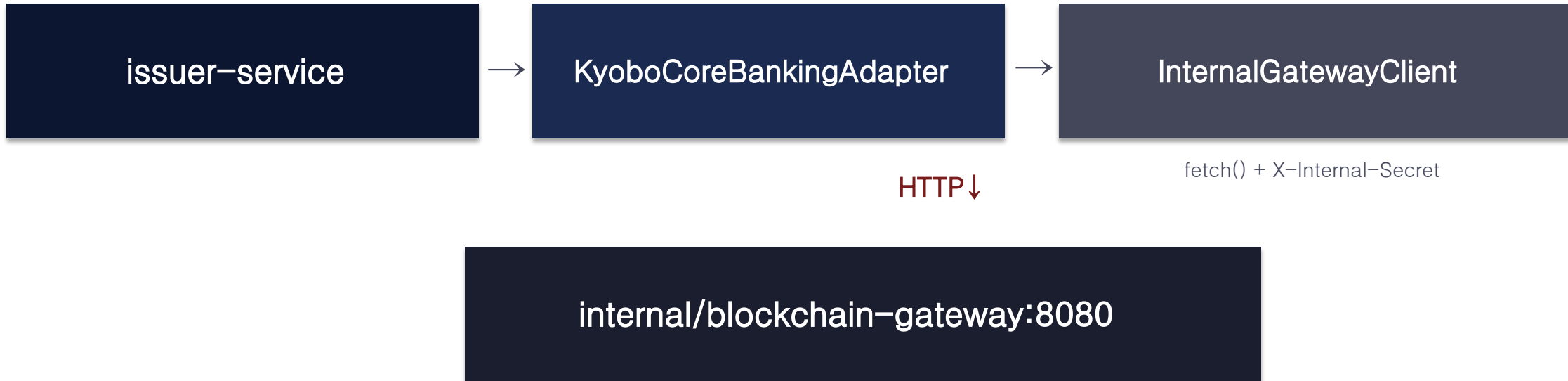


blockchain/ typechain 타입도 import 가능

chain-adapters → blockchain/ ABI 타입 import
→ 컨트랙트 ABI 변경 즉시 컴파일 에러

import 방향 = 단방향. 역방향은 컴파일 자체를 막는다.

런타임 HTTP 호출 방향 — 네트워크



핵심

TypeScript import = 컴파일 타임 / HTTP 호출 = 런타임. 같은 레포에 있어도 TS ↔ Java는 HTTP만.

import vs HTTP — 완전히 다른 개념

TypeScript import (컴파일 타임)

- 같은 프로세스 안 타입 공유
- 빌드 시 번들에 포함
- 네트워크 없음 / 메모리 공유
- internal/ Java는 import ❌
- 오류: 빌드 단계에서 발견

HTTP 호출 (런타임)

- 서로 다른 프로세스 / 서버
- JSON으로만 데이터 교환
- 타입 공유 없음
- DMZ → 내부망 :8080 연결
- 오류: 런타임에 발견

같은 레포 안에 있어도 Node.js ↔ Java는 HTTP만. import 절대 불가.

역방향 의존 — 순환 참조 메커니즘

가정: chain-adapters가 vasp를 import

빌드 순서 결정 불가

vasp 빌드 → chain-adapters 필요
chain-adapters 빌드 → vasp 필요

누구를 먼저 빌드하나?
→ 결정 불가 → 빌드 실패

npm 에러

ERESOLVE unable to resolve dependency tree
Cyclic dependency detected
Cannot find module '@kyobo/vasp'
→ npm install 자체가 실패

역방향은 당장 코드만 보면 관찰아 보임. 빌드해봐야 에러 발견.

역방향 의존 — 변경이 전체로 퍼진다

chain-adapters가 vasp를 알면 — 변경의 파급 효과

연쇄 수정 발생

chain-adapters 수정

- vasp 코드도 바뀌어야 할 수 있음
- issuer-service도 바뀌어야 할 수 있음
- 결국 전체를 다 건드리게 됨

단방향이면

chain-adapters 수정

- IBlockchainAdapter 인터페이스 유지
- vasp, issuer-service 영향 없음
- 변경 범위 = chain-adapters 패키지만

의존 방향이 단방향 → 변경 범위가 한 패키지로 격리.

단방향 의존의 장점 ① — 교체

체인 교체

EVMAAdapter → XRPLAdapter

수정: chain-adapters만

→ IBlockchainAdapter 계약 유지

→ vasp 무변경

→ issuer-service 무변경

VASP 교체

ExternalVASPAdapter (월렛 원)

→ FireblocksVASPAdapter

수정: vasp만

→ IVASPAdapter 계약 유지

→ issuer-service 무변경

인터페이스 = 교체 지점. 계약만 지키면 내부 구현은 자유.

단방향 의존의 장점 ② — 테스트

MockBlockchainAdapter

```
class MockBlockchainAdapter
  implements IBlockchainAdapter {
    async mintNFT(p) {
      return { status: 'success', ... };
    }
  }
}
```

→ 실제 Sepolia 없이 단위 테스트

StubCoreBankingAdapter

```
class StubCoreBankingAdapter
  implements ICoreBankingAdapter {
    // 인메모리 저장
  }
```

→ Java 서버 없이 로컬 실행

→ 빠른 테스트 (네트워크 없음)

인터페이스가 경계 → Stub/Mock 주입으로 독립 테스트 가능.

단방향 의존의 장점 ③ — 확장

새 체인 추가

```
class SolanaAdapter
  implements IBlockchainAdapter {
    readonly chainType = 'SOLANA';
    async mintNFT(p) { ... }
  }
```

- 기존 코드 0줄 수정
- vasp / issuer-service 무관

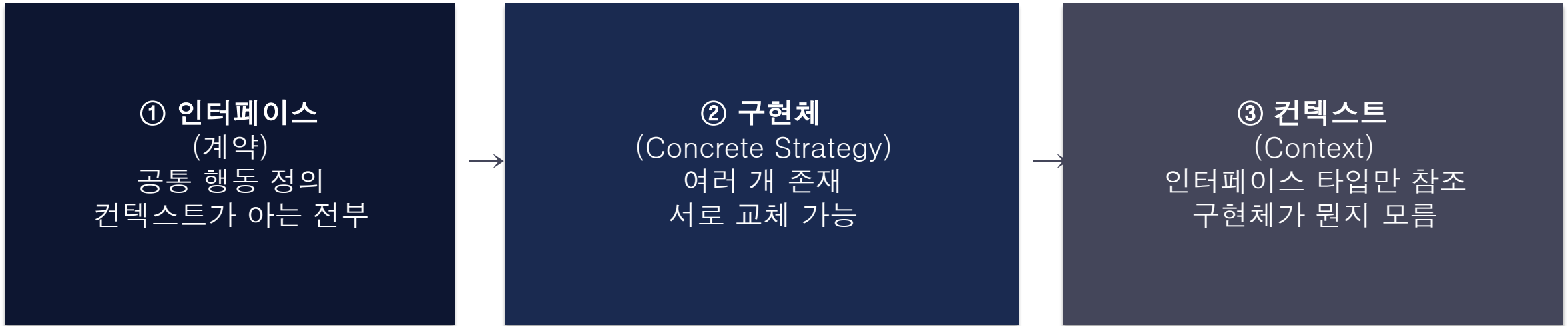
새 VASP 추가

```
class KyoboVASPAdapter
  implements IVASPAdapter {
    // Phase 3 직접 키 관리
  }
```

- vasp 패키지만 수정
- issuer-service 무변경

OCP (Open-Closed Principle): 확장에는 열려 있고 수정에는 닫혀 있다.

Strategy Pattern — 3요소

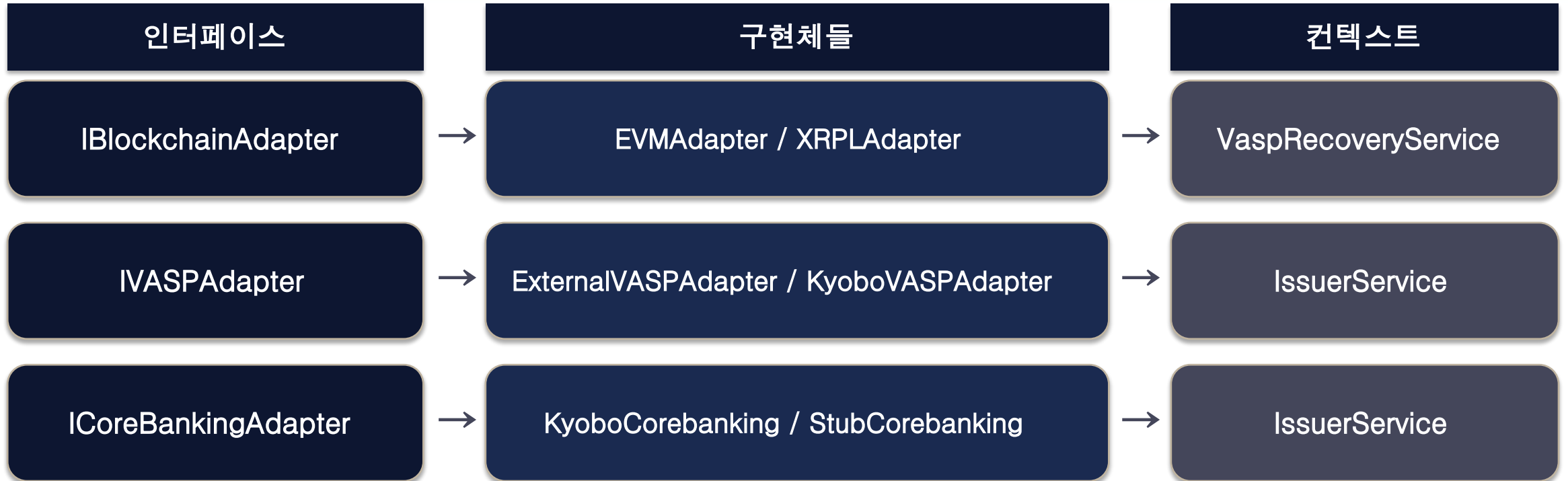


장점

컨텍스트 코드 변경 없이 → 구현체만 교체
테스트 시 → Mock 구현체 주입
새 구현체 추가 → 기존 코드 0줄 수정

한 번 이해하면 이 레포의 모든 레이어가 동일하게 읽힌다.

Strategy Pattern — 이 레포에서 3번 반복



3번 반복. 인터페이스가 바뀌지 않는 한 구현체 교체는 자유.

IBlockchainAdapter — 인터페이스

interface IBlockchainAdapter

```
readonly chainId: string;  
readonly chainType: 'EVM' | 'XRPL' | 'BFT' | 'UTXO';  
isConnected(): Promise<boolean>;  
getBlockNumber(): Promise<number>;  
mintNFT(p: MintParams): Promise<TransactionReceipt>;  
mintNFTBatch(p: MintBatchParams): Promise<TransactionReceipt>;  
burnNFT(p: BurnParams): Promise<TransactionReceipt>;  
getBalance(addr, op, tokenId): Promise<bigint>;  
call(p: ContractCallParams): Promise<unknown>;  
sendTransaction(p): Promise<TransactionReceipt>;  
getReceipt(txHash: string): Promise<TransactionReceipt | null>;  
queryEvents(addr, topics, event, from, to): Promise<ChainEvent[ ]>;
```

S14 실습: 이 인터페이스를 구현하는 4개 어댑터를 같은 코드로 동작시킨다.

EVMAdapter — IBlockchainAdapter 구현체

EVMAdapter 구조

```
class EVMAdapter implements IBlockchainAdapter
{
  readonly chainId: string;
  readonly chainType = 'EVM' as const;

  constructor(config: {
    rpcUrl: string;
    chainId: string;
    privateKey?: string;
  }) { ... }
```

mintNFT 구현

```
async mintNFT(p: MintParams) {
  const tx = await contract.mintBatch(
    p.to, [p.tokenId], [1], '0x'
  );
  const r = await tx.wait();
  return {
    txHash: tx.hash,
    gasUsed: r.gasUsed, status: 'success',
  };
}
```

ethers.js v6 기반. privateKey 없으면 read-only 모드.

IBlockchainAdapter — 컨텍스트는 구현체를 모른다

컨텍스트 (VaspRecoveryService)

```
class VaspRecoveryService {  
    constructor(  
        private blockchain: IBlockchainAdapter  
    ) {}  
  
    async checkTx(txHash: string) {  
        // EVMAdapter인지 XRPLAdapter인지 모름  
        return this.blockchain.getReceipt(txHash);  
    }  
}
```

주입 방식

```
// 운영  
new VaspRecoveryService(new EVMAdapter({...}))  
  
// 테스트  
new VaspRecoveryService(new MockAdapter())  
  
// XRPL 교체  
new VaspRecoveryService(new XRPLAdapter({...}))
```

생성자 주입(DI). VaspRecoveryService 코드는 하나, 어댑터는 교체 가능.

IVASPAdapter — 비즈니스 레이어 인터페이스

interface IVASPAdapter

```
requestMint(params: {  
  requestId: string;    // Idempotency-Key  
  tokenId: bigint;  
  recipientWallet: string;  
  idempotencyKey: string;  
}): Promise<VaspMintResponse>;  
getMintStatus(requestId: string): Promise<MintStatus>;  
subscribeWebhook(handler: (event) => void): void;
```

IBlockchainAdapter와 차이

IVASPAdapter: requestId, Idempotency-Key 관리 (비즈니스 레이어)

IBlockchainAdapter: txHash, gasUsed, chainType 관리 (기술 레이어)

변경 이유가 다름 → 인터페이스 분리. SRP.

IVASPAdapter vs IBlockchainAdapter — SRP

IVASPAdapter — 왜 바뀌는가

- 규제 변화 (VASP 사업자 변경)
- 월렛원 → Fireblocks 교체
- 비즈니스 정책 변경
- Idempotency, requestId 정책
→ 비즈니스 이유

IBlockchainAdapter — 왜 바뀌는가

- 체인 기술 변경 (EVM → XRPL)
- 새 체인 지원 추가
- ethers.js 버전 업그레이드
- ABI 인터페이스 변경
→ 기술 이유

SRP: 변경 이유(reason to change)가 다르면 인터페이스를 분리해야 한다

두 레이어를 하나로 합치면? 규제 변경 때마다 체인 코드도 건드려야 함.

ICoreBankingAdapter — 언어 경계를 넘는 계약

interface ICoreBankingAdapter

```
getUserAccount(userId: string): Promise<UserAccount | null>;  
recordNftHolding(params: {  
  userId: string; tokenId: number;  
  contractAddr: string; chainId: number;  
  acquiredAt: string; onChainTx: string;  
}): Promise<void>;  
recordAuditLog(entry: { ... }): Promise<void>;  
notifyReward(notification: RewardNotification): Promise<void>;
```

왜 이 인터페이스가 필요한가

DMZ(TS)는 Java 클래스 import 불가 → 인터페이스 + JSON DTO로만 계약
→ 런타임에 HTTP 연결 / InternalGatewayClient가 계약 이행

Java 없이 Stub으로 M1~M5 전체 실습 가능. M8에서 실제로 교체.

KyoboCoreBankingAdapter — 운영 구현체 (HTTP)

KyoboCoreBankingAdapter 구조

```
class KyoboCoreBankingAdapter
implements ICoreBankingAdapter {
    constructor(private gw: InternalGatewayClient) {}

    async recordNftHolding(p) {
        return this.gw.recordNftHolding(p.userId, {
            tokenId: p.tokenId,
            contractAddr: p.contractAddr,
            chainId: p.chainId,
            onChainTx: p.onChainTx,
        });
    }
}
```

InternalGatewayClient를 주입받아 Java Gateway :8080 HTTP 호출.

StubCoreBankingAdapter — 인메모리 Stub

Stub 구현체

```
class StubCoreBankingAdapter
implements ICoreBankingAdapter {
    private users = new Map();
    private holdings: unknown[] = [];

    seedUser(u: UserAccount) {
        this.users.set(u.userId, u); }
    async getUserAccount(id) {
        return this.users.get(id) ?? null; }
    async recordNftHolding(p) {
        this.holdings.push(p); }
}
```

검증 메서드

```
// ICoreBankingAdapter에 없음
// Stub 전용 API

stub.getHoldings()
stub.getAuditLog()
stub.getUsers()

// 테스트 검증 예시
assert(
    stub.getAuditLog().length === 1
);
```

Java 없이 동작. IssuerService는 어느 구현체인지 모름.

InternalGatewayClient — 역할과 위치

역할

- DMZ에서 Java Gateway로 HTTP 호출
- 타입 안전한 메서드 형태로 API 제공
- X-Internal-Secret 인증 헤더 처리
- 에러를 InternalGatewayError로 변환

URL 하드코딩 vs 클라이언트

- ❌ `fetch('/api/internal/users/...')`
 - URL 오타, 타입 없음
- ✅ `gateway.recordNftHolding(userId, req)`
 - 타입 체크 + IDE 자동완성
 - URL 변경 = 클라이언트만 수정

URL을 알 필요 없음. 메서드 이름과 타입만 보면 됨.

InternalGatewayClient — 생성자

생성자

```
class InternalGatewayClient {  
    constructor(private config: {  
        baseUrl: string; // 'http://blockchain-gateway:8080'  
        secret: string; // INTERNAL_GATEWAY_SECRET 환경변수  
    }) {}  
}
```

환경별 baseUrl

로컬: 'http://localhost:8080'
Docker: 'http://blockchain-gateway:8080'
운영: 'http://blockchain-gw.internal:8080'

secret 주입

환경변수: INTERNAL_GATEWAY_SECRET
하드코딩 금지
Stub 사용 시 이 클라이언트 불필요

config를 생성자로 주입 → 환경별 분리. 테스트 시 Stub 어댑터 사용.

request() — private HTTP 처리 메서드

request() 구현

```
private async request<T>(method: string, path: string, body?: unknown): Promise<T> {  
  const res = await fetch(`${this.config.baseUrl}${path}`, {  
    method,  
    headers: {  
      'Content-Type': 'application/json',  
      'X-Internal-Secret': this.config.secret,  
    },  
    body: body ? JSON.stringify(body) : undefined,  
  });  
  if (!res.ok)  
    throw new InternalGatewayError(method, path, res.status);  
  return res.json();  
}
```

모든 public 메서드가 이 request()를 호출. 인증·에러처리 중앙 집중.

X-Internal-Secret — 내부망 인증 헤더

인증 방식

'X-Internal-Secret': this.config.secret

- 내부망 방화벽이 외부 접근 차단
- DMZ만 :8080 TCP 연결 가능
- Phase 1 보안 수준으로 충분

Phase 1이 충분한 이유

- 교보 내부망 = 이미 방화벽으로 격리
- 외부에서 :8080 접근 불가
- 시크릿 유출 = 내부망 침투 후에야 가능
- 구현 단순 → 빠른 개발 가능

보안은 위협 모델에 맞게. 과잉 설계는 개발 속도를 낭비.

InternalGatewayClient — API 메서드 목록

메서드	Java endpoint
getUserAccount(userId)	GET /api/internal/users/{userId}
recordNftHolding(userId, req)	POST /api/internal/users/{userId}/nft-holdings
recordAuditLog(req)	POST /api/internal/audit-log
notifyReward(req)	POST /api/internal/rewards/notify

Java Controller의 엔드포인트와 1:1 대응. TS 메서드 = Java API.

TypeScript DTO ↔ Java record 매핑

TypeScript (GatewayNftHoldingRequest)

tokenId: number

contractAddr: string

chainId: number

acquiredAt: string (ISO-8601)

onChainTx: string

Java record (NftHoldingRequest)

Long tokenId

String contractAddr

Integer chainId

Instant acquiredAt (자동 파싱)

String onChainTx

Java Instant = ISO-8601 자동 파싱. `new Date().toISOString()` 그대로.

bigint → Java Long — overflow 주의

ERC-1155 tokenId: uint256 → bigint / Java Long 최대 = $2^{63} - 1$

overflow 조건

tokenId > $2^{63} - 1$
= 9,223,372,036,854,775,807
→ Java Long에서 음수로 표현
→ 데이터 손상

Phase 1에서 안전한 이유

tokenId = 순차 증가 방식
→ 실제로 2^{63} 초과 없음
→ 단, 설계 제약으로 기록 필요
→ M4에서 재검토

uint256 전체 범위 지원이 필요하다면 String으로 전달 후 Java BigInteger 파싱.

인증 전략 — Phase별 위협 모델 변화

Phase 1 (현재)

X-Internal-Secret
공유 시크릿 헤더

- 내부망 방화벽으로 보호
- 구현 단순
- 위협 모델 충분
- 빠른 개발

Phase 2 (향후)

mTLS
상호 인증 TLS

- 클라이언트 인증서
- 양방향 검증
- 자동 로테이션
- 인증서 관리 필요

Phase 3+ (향후)

OAuth2 Client Credentials
토큰 기반

- 권한 세분화
- 감사 추적 강화
- Auth Server 필요
- 복잡도 증가

보안은 과잉이 문제. Phase 1 내부망에서 mTLS / OAuth2는 과잉 투자.

Stub 실습 ① — seedUser + IssuerService 주입

Java 없이 전체 실습 — 코드

```
// 1. Stub 초기화 + seed
const coreBanking = new StubCoreBankingAdapter();
coreBanking.seedUser({
  userId: 'user-001', accountId: 'acc-001',
  walletAddr: '0xf39Fd6e51aad...',
  status: 'active',
});

// 2. IssuerService에 Stub 주입
const issuer = new IssuerService(coreBanking, vasp, ledger);
```

IssuerService는 coreBanking이 Stub인지 실제인지 모름.

Stub 실습 ② — issue() 실행 + 결과 검증

issue() 실행

```
// 3. 발행 실행
const receipt = await issuer.issue({
  userId: 'user-001',
  tokenId: 3n,
  activityType: 'COURSE_COMPLETE',
});

console.log(receipt.txHash);
console.log(receipt.status); // 'success'
```

Stub에서 직접 검증

```
// 4. Stub 검증 메서드로 확인
const log = coreBanking.getAuditLog();
assert(log.length === 1);

const h = coreBanking.getHoldings();
assert(h[0].tokenId === 3);
assert(h[0].userId === 'user-001');
```

Java Gateway 없이 로컬에서 전체 발행 흐름 테스트 완료.

실습 실수 ① — import 경로

❌ 상대 경로 사용

```
import { IVASPAAdapter }  
  from '../..../packages/vasp/src/interfaces'
```

- 파일 이동 시 경로 깨짐
- IDE 자동완성 불안정
- 타입 에러 추적 어려움

✅ 패키지 이름 사용

```
import { IVASPAAdapter }  
  from '@kyobo/vasp'
```

- 파일 이동해도 안전
- IDE 자동완성 완벽 동작
- 패키지 내부 구조 변경 무관

npm workspaces = @kyobo/* 이름으로 import. 상대 경로 쓸 일 없음.

실습 실수 ② — bigint 직렬화

❌ 잘못된 bigint 처리

```
// JSON 직렬화 시도
JSON.stringify({ tokenId: 3n })
→ TypeError: Cannot serialize BigInt

// 비교 연산 실수
if (tokenId == 3) { ... } // 위험
if (tokenId === 3) { ... } // 항상 false
```

✅ 올바른 bigint 처리

```
// JSON 전달 시
JSON.stringify({ tokenId: tokenId.toString() })

// 비교
if (tokenId === 3n) { ... } // 올바름

// number로 변환 (range 주의)
Number(tokenId) // 2^53 이하만 안전
```

ERC-1155 tokenId = uint256 = bigint. 직렬화할 때 반드시 .toString().

실습 실수 ③ — 역방향 import

❌ 역방향 import

```
// chain-adapters에서  
import { IVASPAdapter } from '@kyobo/vasp'  
→ 순환 참조 → 빌드 실패  
  
// shared에서  
import { EVMAAdapter } from  
'@kyobo/chain-adapters'  
→ shared가 최하단 아님 → 다른 패키지 못 씀
```

✅ 해결 방법

공통 타입 → @kyobo/shared로 이동

```
// chain-adapters에서  
import { CommonType } from '@kyobo/shared'  
→ shared는 아무것도 import 안 함  
→ 순환 참조 불가
```

역방향 감지: 빌드 에러 ERESOLVE. 공통 타입은 shared로.

S3 핵심 3원칙

① 모노레포

@kyobo/* import
통합 typecheck
버전 일관성
원자적 변경

② Dependency Rule

안쪽으로만 의존
역방향 = 순환 참조
shared = 최하단
인터페이스 = 경계

③ Strategy Pattern

인터페이스 + 구현체
3번 반복
교체·테스트·확장
컨텍스트는 모름

모노레포: 타입 보장 / Dependency Rule: 파급 제한 / Strategy: 교체 지점

S3 정리

개념	핵심	모들
모노레포	@kyobo/* import / 통합 typecheck	M1
3영역 분리	blockchain / dmz / internal	M4, M6
6개 패키지	chain-adapters / vasp / core-banking / compliance / ...	M2~M7
Dependency Rule	안쪽으로만 의존 / 역방향 = 순환 참조	코드리뷰
import vs HTTP	컴파일타임 타입공유 vs 런타임 데이터교환	M4
Strategy Pattern	인터페이스+구현체+컨텍스트 / 3번	M3
InternalGatewayClient	TS→Java HTTP / X-Internal-Secret	M4
bigint 주의	uint256→bigint / JSON 시 .toString()	전 모들
Phase별 인증	Secret → mTLS → OAuth2	M4
Stub 전략	Java 없이 M1~M5 실습	M4